Universidad de Antioquia
Departamento de Electronica
Informatica II

Desafío 2:

UdeaWorldCup 2026

Andres David Guerrero Rivera
Mauricio Reyes Rosero

26 de abril de 2026

1. Introducción

Se implementa un sistema capaz de simular el torneo mundial de fútbol 2026 mediante programación orientada a objetos en C++. El problema no se limita a la representación de datos, sino que implica modelar un conjunto de entidades interdependientes cuya interacción evoluciona a lo largo del tiempo.

Este sistema coordina múltiples componentes: carga de datos, organización de equipos, simulación de eventos y actualización de estadísticas. Cada decisión de diseño impacta directamente la coherencia del modelo y la estabilidad del sistema durante la ejecución.

2. Análisis del Problema

2.1. Descripción general

El torneo mundial 2026 se organiza en dos grandes fases:

1. **Fase de grupos:** 48 equipos distribuidos en 12 grupos de 4. Cada equipo disputa 3 partidos (todos contra todos dentro del grupo). Clasifican el primero, el segundo y los 8 mejores terceros de los 12 grupos, dando un total de 32 clasificados.
2. **Fase eliminatoria:** Se desarrollan las rondas de dieciseisavos, octavos, cuartos, semifinales, partido por el tercer puesto y final. En cada partido eliminado queda fuera del torneo.

2.2. Clases

Entidad	Clase C++	Responsabilidad
Selección	`Equipo`	**Representar una selección con su plantilla y estadísticas**
Jugador	`Jugador`	**Métricas individuales acumulativas por torneo**
Partido	`Partido`	**Instancia donde se generan eventos y se actualizan datos**
Grupo	`Grupo`	**Contenedor de cuatro equipos en la fase inicial**
Bombo	`Bombo`	**Agrupamiento por ranking para el sorteo**
Est. históricas	`EstadisticasEquipo`	**Historial previo (influye en simulación)**
Est. torneo	`EstadisticasTorneoEquipo`	**Estado acumulado durante el torneo**
Est. jugador	`EstadisticasJugador`	**Métricas individuales durante el torneo**

Tabla 1: Clases principales

2.3. Separación de estadísticas

Una decisión de modelado fundamental fue separar las estadísticas en dos niveles independientes:

Diseño: dos capas de estadísticas

Estadísticas históricas (`EstadisticasEquipo`): reflejan el rendimiento acumulado de la selección en competencias anteriores. Se cargan desde el CSV y sirven como parámetros de entrada al modelo probabilístico de simulación.

Estadísticas del torneo (`EstadisticasTorneoEquipo`): comienzan en cero al inicio del torneo y se actualizan partido a partido. Son las que determinan la clasificación en la tabla de posiciones.

Esta separación garantiza que el comportamiento histórico influya en la simulación sin distorsionar los resultados del torneo en curso, evitando inconsistencias entre el estado inicial y la evolución del sistema.

3. Arquitectura y Diseño

3.1. Organización modular

El sistema se divide en módulos con responsabilidades claramente delimitadas:

Módulo	Clases principales	Responsabilidad
Datos	`GestorDatos`	**Lectura desde CSV y escritura de históricos**
Dominio	`Equipo`, `Jugador`	**Representación de las entidades principales**
Estadísticas	`EstadisticasEquipo`, `EstadisticasTorneoEquipo`, `EstadisticasJugador`	**Métricas en dos niveles (histórico y torneo)**
Simulación	`Partido`, `Calendario`	**Generación y ejecución de encuentros**
Organización	`Grupo`, `Bombo`	**Distribución de equipos**
Clasificación	`TablaClasificacion`, `FilaClasificacion`	**Ordenamiento de resultados por grupo**
Eliminatorias	`Eliminatorias`	**Gestión de la fase de playoffs**
Estructuras	`Lista<T>`, `Nodo<T>`	**Manejo de memoria dinámica genérica**
Medición	`MedidorRecursos`	**Conteo de iteraciones y uso de memoria**

Tabla 2: Módulos del sistema

3.2. Diagrama de relaciones entre clases

Las relaciones de composición y dependencia entre las clases principales son las siguientes:

Clase origen	Relación	Clase destino
`Equipo`	**contiene**	`Lista<Jugador>`
`Equipo`	**contiene**	`EstadisticasEquipo`
`Equipo`	**contiene**	`EstadisticasTorneoEquipo`
`Jugador`	**contiene**	`EstadisticasJugador`
`Grupo`	**contiene**	`Lista<Equipo>`
`Bombo`	**contiene**	`Lista<Equipo>`
`Partido`	**referencia**	`Equipo*`, `Equipo*`
`Partido`	**contiene**	`Lista<Jugador*>`
`TablaClasificacion`	**contiene**	`Lista<FilaClasificacion>`
`FilaClasificacion`	**contiene**	`Equipo` **(copia)**
`Eliminatorias`	**contiene**	`Lista<Equipo>`

3.3. Flujo general de ejecución

El programa principal en `main.cpp` coordina el flujo completo del torneo en las siguientes etapas:

1. Carga de equipos desde el archivo CSV mediante `GestorDatos`.
2. Ordenamiento por ranking FIFA y ajuste de la posición del equipo anfitrión (Estados Unidos).
3. Distribución en cuatro bombos según el ranking.
4. Conformación de 12 grupos (un equipo de cada bombo por grupo).
5. Generación del calendario de fase de grupos con `Calendario`.
6. Simulación de los 72 partidos de la fase de grupos.
7. Clasificación de equipos y obtención de los 32 clasificados.
8. Ejecución de la fase eliminatoria ronda a ronda hasta la final.
9. Exportación de históricos de equipos y jugadores a CSV.

Cada etapa incluye medición de iteraciones y memoria mediante `MedidorRecursos`.

4. Algoritmos Implementados

4.1. Distribución en bombos y conformación de grupos

Los equipos se ordenan de menor a mayor valor de ranking FIFA. El equipo anfitrión (Estados Unidos) se ubica forzosamente en la primera posición. Luego se distribuyen en cuatro bombos de 12 equipos cada uno:

- **Bombo 1:** posiciones 1–12 (los mejores equipos).
- **Bombo 2:** posiciones 13–24.
- **Bombo 3:** posiciones 25–36.
- **Bombo 4:** posiciones 37–48.

Los grupos se conforman asignando el equipo i de cada bombo al grupo i (para $i = 0, 1, \ldots, 11$). Esto garantiza heterogeneidad en cada grupo y evita la concentración de equipos de alto o bajo ranking.

4.2. Generación del calendario de fase de grupos

Para cada grupo se generan tres jornadas siguiendo un esquema fijo de enfrentamientos:

Jornada	Partido A	Partido B
1	**Equipo 1 vs Equipo 2**	**Equipo 3 vs Equipo 4**
2	**Equipo 1 vs Equipo 3**	**Equipo 2 vs Equipo 4**
3	**Equipo 1 vs Equipo 4**	**Equipo 2 vs Equipo 3**

Tabla 3: Calendario de fase de grupos

Cada equipo disputa exactamente 3 partidos, uno contra cada rival del grupo, sin repetición. Las fechas se asignan a partir del 20 de junio de 2026, calculando el día en función del índice del partido dentro del torneo.

4.3. Modelo probabilístico de simulación de partidos

Para cada equipo se calcula la tasa esperada de goles λ mediante:

$$\lambda_i = \mu \cdot \left(\frac{GF_i/PJ_i}{\mu}\right)^{\alpha} \cdot \left(\frac{GC_j/PJ_j}{\mu}\right)^{\beta} \tag{1}$$

donde:

- $\mu = 1{,}35$: promedio global de goles por partido en competiciones de alto nivel.
- GF_i/PJ_i: promedio de goles a favor del equipo i según su historial.
- GC_j/PJ_j: promedio de goles en contra del rival j según su historial.
- $\alpha = 0{,}6$: peso del poder ofensivo del equipo.
- $\beta = 0{,}4$: peso de la debilidad defensiva del rival.

Si un equipo no tiene partidos previos registrados ($PJ = 0$), se asume $\lambda = \mu$, garantizando un comportamiento razonable incluso con datos faltantes.

En partidos de eliminatoria (donde no puede haber empate), si los goles quedan igualados se otorga un gol adicional al equipo con mejor ranking FIFA, simulando la definición por penales.

4.4. Selección de titulares

Para cada partido se seleccionan 11 jugadores distintos por equipo mediante muestreo aleatorio sin reemplazo. El algoritmo mantiene una lista de camisetas ya seleccionadas y evita duplicados

4.5. Simulación de eventos individuales

Cada jugador titular acumula minutos jugados (90 en fase de grupos, posiblemente 120 en eliminatorias) y puede recibir tarjetas y cometer faltas según probabilidades independientes:

Evento	Probabilidad base	Probabilidad condicional
Primera tarjeta amarilla	**6.0 %**	—
Segunda amarilla (= roja)	**1.15 %**	**Si ya tiene amarilla**
Primera falta	**13.0 %**	—
Segunda falta	**2.75 %**	**Si ya tiene falta**
Tercera falta	**0.70 %**	**Si ya tiene dos faltas**

Tabla 4: Probabilidades de eventos individuales por partido

Los goles se distribuyen aleatoriamente entre los titulares del equipo, actualizando las estadísticas individuales de cada jugador.

4.6. Criterios de clasificación

El ordenamiento en las tablas de posiciones sigue los criterios reglamentarios de la FIFA, implementados mediante la sobrecarga del operador `<` en `FilaClasificacion`:

1. Mayor cantidad de puntos (victoria = 3, empate = 1, derrota = 0).
2. Mayor diferencia de goles ($GF - GC$).
3. Mayor cantidad de goles a favor (GF).
4. Mejor posición en el ranking FIFA (como desempate final).

4.7. Generación de dieciseisavos de final

La conformación de los cruces en dieciseisavos respeta la restricción de que dos equipos del mismo grupo no pueden enfrentarse en esa ronda. El algoritmo asigna:

- Los 8 primeros de grupo contra los 8 mejores terceros.
- Los 4 últimos primeros de grupo contra los 4 mejores segundos restantes.
- Los 8 segundos de grupo entre sí.

Cuando una asignación natural generaría un cruce del mismo grupo, el algoritmo busca mediante `buscarCompatible()` el siguiente candidato válido en la lista ordenada e intercambia posiciones.

5. Estructuras de Datos

5.1. Lista enlazada genérica

La estructura de datos fundamental del sistema es una lista enlazada simple genérica implementada como clase plantilla `Lista<T>`. Cada nodo de la lista, representado por `Nodo<T>`, almacena:

- Un dato de tipo `T` (copia por valor).
- Un puntero al siguiente nodo (`Nodo<T>* ptrNext`).

La clase `Lista<T>` expone las siguientes operaciones principales:

Operación	Complejidad	Descripción
`agregar(e, i)`	$O(n)$	**Inserta en la posición i**
`consultar(i)`	$O(n)$	**Accede al elemento en posición i**
`tamano()`	$O(1)$	**Retorna el número de elementos**
`esVacia()`	$O(1)$	**Verifica si la lista está vacía**
`primero()`	$O(1)$	**Accede al primer elemento**
`ultimo()`	$O(n)$	**Accede al último elemento**
`ordenar()`	$O(n^2)$	**Bubble sort usando operador <**
`memoryUsage()`	$O(1)$	**Retorna memoria aproximada usada**

Tabla 5: Operaciones de `Lista<T>`

5.2. Gestión de memoria dinámica

El constructor de copia realiza una **copia profunda**: recorre la lista original nodo a nodo e inserta nuevas copias de cada elemento, garantizando que ambas listas sean completamente independientes en memoria.

5.3. Medición de recursos

La clase `MedidorRecursos` implementa un contador estático global de iteraciones que se incrementa en cada operación relevante (recorrido de nodos, acceso a elementos, simulación de eventos). Esta medición permite analizar empíricamente la complejidad de cada etapa del torneo.

Adicionalmente, `Lista<T>` mantiene un miembro estático `totalMemory` que se actualiza al crear o destruir nodos, permitiendo estimar la memoria dinámica total utilizada.

5.4. Tipos de listas usadas en el sistema

El sistema instancia `Lista<T>` con los siguientes tipos a lo largo de su ejecución:

- `Lista<Equipo>`: almacena todos los equipos del torneo, así como los equipos de cada grupo, bombo y ronda eliminatoria.
- `Lista<Jugador>`: plantilla de 23 jugadores por equipo.
- `Lista<Partido>`: partidos de cada fase.
- `Lista<Grupo>`: los 12 grupos del torneo.
- `Lista<FilaClasificacion>`: filas de cada tabla de posiciones.
- `Lista<Jugador*>`: punteros a los 11 titulares de un partido.
- `Lista<int>`: números de camiseta de goleadores.
- `Lista<string>` y `Lista<char>`: uso auxiliar.

6. Problemas Afrontados y Soluciones

6.1. Copias profundas en clases compuestas

Problema: La clase `Equipo` contiene internamente una `Lista<Jugador>` y dos objetos de estadísticas. Al copiar un equipo (por ejemplo, al insertarlo en un grupo o en una `FilaClasificacion`), era necesario garantizar que la copia fuera completamente independiente.

Solución: Implementación explícita del constructor de copia y del operador de asignación en todas las clases que poseen miembros con memoria dinámica (`Equipo`, `Lista<T>`, `Partido`). La lista enlazada realiza copia profunda recorriendo todos los nodos.

6.2. Punteros en la clase Partido

Problema: `Partido` almacena punteros `Equipo*` en lugar de copias, para permitir que los resultados actualicen directamente las estadísticas del equipo original. Sin embargo, esto introduce el riesgo de punteros colgantes si el equipo al que apuntan es destruido o movido en memoria.

Solución: El ciclo de vida de los equipos se gestiona en `main.cpp`. Los equipos de rondas eliminatorias se almacenan en el miembro `equiposRonda` de `Eliminatorias`, garantizando que los punteros sean válidos mientras el partido existe. Se incluyeron verificaciones `nullptr` en todos los métodos que acceden a los punteros.

6.3. Consistencia de estadísticas en múltiples niveles

Problema: Las estadísticas del equipo existen en tres objetos distintos: `EstadisticasEquipo` (histórico), `EstadisticasTorneoEquipo` (torneo) y `FilaClasificacion` (clasificación). Mantener la coherencia entre ellos durante y después de cada partido fue un desafío de diseño.

Solución: El método `Partido::actualizarEstadisticasEquipos()` centraliza toda la lógica de actualización al finalizar cada simulación, modificando ambos objetos de estadísticas del equipo a través de los punteros. La `FilaClasificacion` se recarga desde el equipo mediante `cargarDesdeEquipo()` al construir cada tabla de posiciones.

7. Medición de Recursos

El sistema incorpora medición de iteraciones y memoria en cada etapa del torneo. La clase `MedidorRecursos` utiliza un contador estático que se reinicia entre etapas, y `Lista<T>::totalMemory` acumula la memoria dinámica asignada mediante `new`.

Las etapas medidas son:

1. Carga de equipos desde CSV.
2. Ordenamiento por ranking y ajuste del anfitrión.
3. Creación de bombos.
4. Conformación de grupos.
5. Generación del calendario de fase de grupos.
6. Simulación de partidos de fase de grupos.
7. Clasificación a dieciseisavos.
8. Cada ronda eliminatoria (dieciseisavos, octavos, cuartos, semifinales, tercer puesto, final).
9. Guardado de históricos.

8. Persistencia de Datos

8.1. Carga desde CSV

La clase `GestorDatos` implementa `cargarEquiposDesdeCSV()`, que lee el archivo de selecciones clasificadas con el siguiente formato (separador `;`):

```
ranking; pais; directorTecnico; federacion; confederacion; GF; GC; PG; PE; PP
```

El método omite las dos primeras líneas (encabezados) y construye un objeto `Equipo` por cada fila válida, inicializando sus estadísticas históricas con los valores leídos.

8.2. Exportación de históricos

Al finalizar el torneo se exportan dos archivos CSV:

- `historico_equipos.csv`: estadísticas acumuladas de cada selección durante el torneo (goles, partidos, tarjetas, faltas).
- `historico_jugadores.csv`: estadísticas individuales de todos los jugadores (minutos, goles, tarjetas, faltas).

Estos archivos permiten alimentar ediciones futuras del torneo con los datos generados, cerrando el ciclo de persistencia del sistema.

9. Conclusiones

El desarrollo del sistema de simulación del Mundial 2026 permitió integrar de manera aplicada los principales conceptos de programación orientada a objetos en C++:

- **Encapsulamiento:** cada clase expone únicamente la interfaz necesaria, ocultando la implementación interna. Las estadísticas, los jugadores y los resultados se modifican exclusivamente a través de métodos definidos.
- **Composición:** las entidades complejas (`Equipo`, `Partido`, `Grupo`) se construyen como composición de clases más simples, favoreciendo la reutilización y el mantenimiento.
- **Plantillas genéricas:** la lista enlazada `Lista<T>` es utilizada con múltiples tipos a lo largo del sistema, evitando duplicación de código y garantizando comportamiento homogéneo.
- **Gestión de memoria dinámica:** la regla de tres (constructor de copia, operador de asignación, destructor) se implementó en todas las clases pertinentes, previniendo fugas y corrupción de memoria.
- **Sobrecarga de operadores:** los operadores `<` y `==` fueron sobrecargados para permitir el ordenamiento y comparación natural de equipos, jugadores y filas de clasificación.
- **Modelado probabilístico:** el uso de distribuciones de Poisson parametrizadas por estadísticas históricas dota al simulador de un comportamiento realista y diferenciado según el nivel de cada selección.